SİVAS CUMHURİYET ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ — WEB PROGRAMLAMA I DERSİ

Vue.js Framework Temelleri

Modern Frontend Geliştirme, Uygulama Mimarisi, Şablon Sözdizimi ve Bileşen Yapısı

HAZIRLAYAN: Web Programlama I Dersi Ekibi

KAPSAM: Hızlı Başlangıç • Vue Uygulaması Oluşturma • Şablon Sözdizimi • Bileşen Temelleri

BÖLÜM 1

Quick Start (Hızlı Başlangıç)

Vue.js'e giriş, temel özellikler, çevrimiçi araçlar, CDN kullanımı, ES Modülleri, Node.js/Vite ve Single-File Components (SFC) mimarisi.

WEB PROGRAMLAMA I — VUE.JS REHBERİ

Vue.js Nedir & Temel Mimarisi

Kademeli (Progressive) Framework

- Vue.js, kullanıcı arayüzleri (UI) inşa etmek için tasarlanmış açık kaynaklı bir JavaScript framework'üdür.
- Kademeli Mimari: Basit bir CDN scripti ile statik HTML sayfalarına kolayca eklenebilir veya devasa SPA uygulamalarına dönüştürülebilir.
- Kolay Öğrenme Eğrisi: HTML, CSS ve standart JS bilgisi olan geliştiriciler tarafından hızla adapte edilebilir.
- Geniş Ekosistem: Resmi yönlendirici (Vue Router), durum yönetimi (Pinia) ve derleme araçları (Vite) ile tam destek sunar.

Temel Çalışma İlkeleri

- Bildirimsel Programlama (Declarative Rendering): Doğrudan DOM manipülasyonu yerine durum (state) ile görünüm bildirimsel olarak bağlanır.
- Reaktivite Sistemi (Reactivity): JavaScript durumundaki değişimler otomatik olarak takip edilir ve DOM en az maliyetle güncellenir.
- Bileşen Mimarisi (Component-Based): Kullanıcı arayüzü bağımsız, modüler ve yeniden kullanılabilir parçalara bölünür.
- Sanal DOM (Virtual DOM): Derleme zamanı optimizasyonları ile yüksek performanslı arayüz güncellemeleri gerçekleştirir.

Çevrimiçi Deneyim ve Test Araçları

Vue Playground (Oyun Alanı)

- Tarayıcı üzerinden doğrudan Vue Tek Dosya Bileşenlerini (SFC) deneme ve hızlı prototip oluşturma olanağı sağlar.
- Kurulum veya Node.js gereksinimi olmadan reaktif kod parçacıklarını anında test etmeye yarar.

JSFiddle & Sade HTML

- Herhangi bir derleme (build) adımı içermeyen en basit başlangıç ortamıdır.
- CDN bağlantısı ile statik HTML dosyalarında hızlıca test yapmayı destekler.

StackBlitz Tarayıcı Ortamı

- Tarayıcı içerisinde eksiksiz bir Node.js ve Vite derleme kurulumu çalıştırır.
- Yerel geliştirme ortamına en yakın çevrimiçi deneyimi sunar.

Scrimba Etkileşimli Eğitim

- Kod düzenleme ve video anlatımını birleştiren etkileşimli öğrenme platformudur.
- İlk Vue uygulamasını adım adım çalıştırmak için önerilen araçlardandır.

CDN ile Kullanım & Küresel Derlem (Global Build)

CDN Entegrasyonu Özellikleri

- Derleme adımı (build step) olmadan doğrudan HTML sayfasına eklenir.
- Script Etiketi: vue.global.js CDN adresi kullanılır.
- Küresel Nesne: Vue tarafından sağlanan tüm API'ler (createApp, ref vb.) küresel 'Vue' nesnesi üzerinden erişilebilir hale gelir.
- Kullanım Alanı: Mevcut arka uç (Laravel, Django, ASP.NET) uygulamalarına reaktif arayüz parçaları eklemek için idealdir.

```
// index.html (Global CDN Örneği)
<!DOCTYPE html>
<html>
<head>
  <!-- Vue 3 Global CDN -->
</head>
<body>
  <div id="app">
    <h1>{{ message }}</h1>
  </div>

  <script>
    const { createApp, ref } = Vue

    createApp({
      setup() {
        const message = ref('Merhaba Vue!')
        return { message }
      }
    }).mount('#app')
  </script>
</body>
</html>
```

ES Modül Derlemesi (ESM) & İçe Aktarma Haritaları

ESM ve Import Maps Mantığı

- Yerel ES Modülleri: Modern tarayıcılarda script type='module' kullanarak CDN üzerinden doğrudan ES modülü yüklenebilir.
- Import Maps (İçe Aktarma Haritaları): Tarayıcıya 'vue' modül adının hangi CDN URL'sine karşılık geldiğini tanımlar.
- Temiz Sözdizimi: Küresel 'Vue' nesnesi yerine doğrudan 'import { createApp } from 'vue'' sözdizimi kullanılır.
- Tarayıcı Desteği: Safari 16.4+ ve tüm modern tarayıcılar tarafından yerel olarak desteklenir.

```
// index.html (ESM & Import Map)
<!DOCTYPE html>
<html>
<head>
  <script type="importmap">
    {
      "imports": {
        "vue": "https://unpkg.com/vue@3/dist/vue.esm-browser.js"
      }
    }
  </script>
</head>
<body>
  <div id="app">{{ count }}</div>

  <script type="module">
    import { createApp, ref } from 'vue'

    createApp({
      setup() {
        const count = ref(10)
        return { count }
      }
    }).mount('#app')
  </script>
</body>
</html>
```

Modüllerin Ayrılması & HTTP Sunucu Zorunluluğu

HTTP Sunucusu Neden Zorunlu?

- file:// Protokol Kısıtlaması: Tarayıcılar güvenlik (CORS) nedeniyle ES modüllerini doğrudan yerel dosya sisteminden (file://) yüklemez.
- HTTP Protokolü (http://): ES modüllerinin çalışması için uygulamanın yerel bir HTTP sunucusu üzerinden sunulması şarttır.
- Sunucu Başlatma Araçları: 'npx serve' veya VS Code 'Live Server' eklentisi ile hızlıca bir HTTP sunucusu başlatılabilir.
- Modül Ayrımı: JavaScript mantığı 'my-component.js' gibi ayrı dosyalara bölünerek modüler mimari kurulabilir.

Vue Ekosistemi & Çerçeveler

- Nuxt: Sunucu Tarafı İşleme (SSR), Statik Site Oluşturma (SSG) ve otomatik yönlendirme sunan tam kapsamlı Vue çerçevesi.
- VitePress: Vue kaynaklı dokümantasyon ve statik içerik siteleri oluşturmak için optimize edilmiş araç.
- Quasar: Tek bir Vue kod tabanından Masaüstü (Electron), Mobil (Capacitor) ve Web uygulaması çıkarma olanağı.
- Seçim Kriteri: Basit sayfalar için CDN; ölçeklenebilir web uygulamaları için Vite/SFC tercihi yapılmalıdır.

Node.js, Vite ve create-vue ile Proje Kurulumu

Üretim Seviyesinde SPA Kurulumu

- Sistem Önkoşulları: Komut satırı kullanımı ve Node.js (^22.18.0 || >=24.12.0) kurulu olmalıdır.
- create-vue Aracı: Vue'nun resmi proje iskeleti oluşturma aracıdır. Vite tabanlı bir mimari kurar.
- İsteğe Bağlı Özellikler: TypeScript, Vue Router, Pinia (durum yönetimi), Vitest ve ESLint yönlendirmeleri sunar.
- Vite Avantajı: Anında modül değiştirme (HMR) ve yıldırım hızında geliştirme sunucusu sağlar.

```
// Komut Satırı (Terminal) Adımları
# 1. Proje oluşturma komutunu çalıştırın
$ npm create vue@latest

# Yönlendirme Seçenekleri:
✓ Project name: ... my-vue-app
✓ Add TypeScript? ... No / Yes
✓ Add Vue Router? ... No / Yes
✓ Add Pinia? ... No / Yes

# 2. Proje dizinine geçin ve bağımlılıkları yükleyin
$ cd my-vue-app
$ npm install

# 3. Geliştirme sunucusunu başlatın
$ npm run dev

# 4. Üretim (Production) derlemesi alın
$ npm run build
```

Tek Dosya Bileşenleri (SFC - Single-File Components)

SFC (.vue) Anatomisi

- Kapsüllenmiş Yapı: HTML şablonu, JavaScript mantığı ve CSS stillerini tek bir .vue dosyasında birleştirir.
- <script setup>: Derleme zamanı dönüştürmesiyle reaktif durum değişkenlerini ve fonksiyonları şablona otomatik sunar.
- <template>: Bileşenin HTML görünümünü ve bildirimsel veri bağlamalarını içerir.
- <style scoped>: 'scoped' eki sayesinde yazılan stillerin yalnızca o bileşene etki etmesini sağlar.

```
// ButtonCounter.vue (SFC Örneği)
<script setup>
import { ref } from 'vue'

// Reaktif durum değişkeni
const count = ref(0)
</script>

<template>
  <button @click="count++">
    Tıklama Sayısı: {{ count }}
  </button>
</template>

<style scoped>
button {
  font-weight: bold;
  padding: 8px 16px;
  background-color: #42b883;
  color: white;
  border: none;
  border-radius: 4px;
}
</style>
```

BÖLÜM SONU ÖZETİ

1. Bölüm Sonu: Quick Start (Hızlı Başlangıç) — Özet ve Değerlendirme

Bölüm 1 Kazanımları ve Temel Saptamalar

- Vue.js, kullanıcı arayüzü geliştirmeyi kolaylaştıran reaktif ve kademeli bir JavaScript framework'üdür.
- Prototipleme ve basit entegrasyonlar için CDN veya ES Modülleri (Import Maps) doğrudan kullanılabilir.
- Üretim seviyesinde Tek Sayfa Uygulamaları (SPA) için Node.js, Vite ve create-vue araçları tercih edilir.
- Tek Dosya Bileşenleri (SFC - .vue), mantık, görünüm ve stilleri tek dosyada modüler olarak kapsüller.

BÖLÜM 2

Creating a Vue Application (Vue Uygulaması Oluşturma)

createApp() fonksiyonu, Kök Bileşen (App.vue), DOM'a yükleme (mount), uygulama yapılandırması, birden fazla uygulama örneği ve global varlık kayıtları.

Uygulama Örneği (Core Application Instance)

createApp() Fonksiyonu

- Başlangıç Noktası: Her Vue uygulaması, createApp() fonksiyonu ile yeni bir uygulama örneği oluşturarak başlar.
- Uygulama Kapsamı: Oluşturulan 'app' örneği, uygulamanın yaşam döngüsünü, global ayarlarını ve bileşen kaydını yönetir.
- Seçenekler Nesnesi: Fonksiyona ilk parametre olarak kök bileşen veya bir seçenekler nesnesi aktarılır.
- Zincirleme Yapı: Metotlar (component, use, directive) zincirleme olarak çağrılabilir.

```
// main.js (Temel Örnek)
import { createApp } from 'vue'

// 1. Uygulama örneğini oluşturun
const app = createApp({
  /* kök bileşen seçenekleri */
  data() {
    return {
      title: 'Vue Uygulamam'
    }
  }
})

// 2. Uygulamayı DOM'a bağlayın
app.mount('#app')
```

Kök Bileşen (Root Component) & Bileşen Ağacı

App.vue ve Ağaç Mimarisi

- Kök Bileşen (Root Component): createApp() fonksiyonuna aktarılan ana bileşendir (genellikle App.vue).
- Hiyerarşik Yapı: Diğer tüm alt bileşenler kök bileşenin altında bir ağaç yapısı (Component Tree) oluşturur.
- Modüler Tasarım: Bir Yapılacaklar (Todo) uygulamasında App -> TodoList -> TodoItem hiyerarşisi kurulur.
- Yeniden Kullanılabilirlik: Her alt bileşen kendi mantık ve görünümünü kapsüller.

```
// Bileşen Hiyerarşi Şeması
App (Root Component)
├─ AppHeader
│  └─ UserProfile
├─ TodoList
│  ├─ TodoItem
│  │  ├─ TodoDeleteButton
│  │  └─ TodoEditButton
│  └─ TodoFilter
└─ AppFooter
   └─ TodoStatistics
```

Uygulamayı Yükleme (.mount) & DOM İçi Şablon

DOM'a Bağlama (Mounting)

- .mount() Metodu: Uygulama örneği .mount() metodu çağrılana kadar ekranda hiçbir şey oluşturmaz.
- Kapsayıcı Argümanı: Gerçek bir DOM öğesi (CSS seçici dizesi '#app') alarak uygulamayı bu öğeye bağlar.
- Sadece En Son Çağrı: Tüm yapılandırmalar ve global kayıtlar tamamlandıktan sonra en son .mount() çağrılmalıdır.
- DOM İçi Kök Şablon: Bileşende 'template' yoksa Vue kapsayıcının innerHTML içeriğini otomatik şablon olarak alır.

```
// index.html & main.js
<!-- index.html -->
<div id="app">
  <!-- DOM İçi Kök Şablon -->
  <button @click="count++">
    Sayaç: {{ count }}
  </button>
</div>

<!-- main.js -->
<script type="module">
  import { createApp, ref } from 'vue'

  const app = createApp({
    setup() {
      const count = ref(0)
      return { count }
    }
  })

  // DOM'a yükle
  app.mount('#app')
</script>
```

Uygulama Yapılandırmaları (app.config)

Global Ayarlar & Hata Yakalama

- app.config Nesnesi: Uygulama düzeyinde yapılandırma seçeneklerini barındıran nesnedir.
- errorHandler: Uygulama içindeki tüm alt bileşenlerde yakalanmayan hataları izleyen global fonksiyon.
- globalProperties: Tüm bileşen şablonlarından erişilebilen global özellik veya yardımcı metod tanımlama.
- Performans İzleme: app.config.performance = true ile geliştirme aşamasında zamanlama ölçümü yapılabilir.

```
// main.js (Yapılandırma Örnekleri)
import { createApp } from 'vue'
import App from './App.vue'

const app = createApp(App)

// Global Hata Yakalayıcı
app.config.errorHandler = (err, instance, info) => {
  console.error('Global Hata Yakalandı:', err)
  console.log('Bileşen Örneği:', instance)
  console.log('Hata Detayı:', info)
}

// Global Yardımcı Değişken Tanımlama
app.config.globalProperties. = 'Vue Dersi'

app.mount('#app')
```

Birden Fazla Uygulama Örneği (Multiple Instances)

Bağımsız Vue Örnekleri

- Aynı Sayfada Çoklu Örnek: Aynı HTML sayfasında birbirinden bağımsız birden fazla createApp() kurulabilir.
- Kapsam İzolasyonu: Her uygulama örneği kendi reaktif durumuna, yapılandırmasına ve bileşen alanına sahiptir.
- Kullanım Senaryosu: Sunucu taraflı işlenen (Laravel, ASP.NET) devasa HTML sayfalarında belirli alanlara (header, widget) reaktivite kazandırmak.
- Temiz Ayrım: Bir örnekte yapılan değişiklik veya global kayıt diğer örneği etkilemez.

```
// Çoklu Örnek Kod Yapısı
<!-- HTML Konteynerleri -->
<div id="header-app"></div>
<div id="footer-app"></div>

<script type="module">
  import { createApp } from 'vue'
  import HeaderComponent from './Header.vue'
  import FooterComponent from './Footer.vue'

  // 1. Uygulama Örneği
  const headerApp = createApp(HeaderComponent)
  headerApp.mount('#header-app')

  // 2. Uygulama Örneği
  const footerApp = createApp(FooterComponent)
  footerApp.mount('#footer-app')
</script>
```

Global Varlık Kaydı: Bileşenler, Yönergeler, Eklentiler

Global Kayıt Metotları

- app.component(): Uygulama genelinde içe aktarılmaya gerek kalmadan tüm şablonlarda kullanılabilen global bileşen kaydeder.
- app.directive(): Tüm bileşenlerde geçerli olan v-focus gibi özel DOM yönergeleri tanımlar.
- app.use(): Pinia (durum yönetimi), Vue Router (yönlendirme) veya UI kütüphaneleri gibi eklentileri yükler.
- Zincirleme Metotlar: app.component().use().mount() şeklinde akıcı biçimde yazılabilir.

```
// main.js (Global Kayıtlar)
import { createApp } from 'vue'
import App from './App.vue'
import BaseButton from './BaseButton.vue'
import router from './router'

const app = createApp(App)

# Global Bileşen Kaydı
app.component('BaseButton', BaseButton)

# Özel Yönerge Kaydı (v-focus)
app.directive('focus', {
  mounted: (el) => el.focus()
})

# Eklenti Entegrasyonu
app.use(router)

app.mount('#app')
```

BÖLÜM SONU ÖZETİ

2. Bölüm Sonu: Creating a Vue Application (Vue Uygulaması Oluşturma) — Özet ve Değerlendirme

Bölüm 2 Kazanımları ve Temel Saptamalar

- Vue uygulamaları her zaman createApp() fonksiyonu ile başlatılan bir core instance ile kurulur.
- Uygulama hiyerarşisi App.vue kök bileşeni ve altına bağlanan bileşen ağacından oluşur.
- .mount('#app') metodu çağrılarak uygulama gerçek bir DOM kapsayıcısına bağlanır.
- app.config üzerinden global hata yakalayıcılar, global bileşenler, yönergeler ve eklentiler yönetilir.

BÖLÜM 3

Template Syntax (Şablon Sözdizimi)

Metin interpolasyonu, v-html, v-bind, JavaScript ifadeleri, dinamik nitelikler, yönergeler, değiştiriciler ve v-if, v-for, v-on, v-model mantığı.

Metin İnterpolasyonu & Ham HTML (v-html)

Veri Bağlama ve HTML Çıktısı

- Bıyık Sözdizimi (Mustache): {{ msg }} biçiminde çift süslü parantez ile veriler düz metin olarak bağlanır.
- Otomatik Reaktivite: Bağlanan veri nesnesi değiştiğinde şablondaki metin otomatik olarak güncellenir.
- Ham HTML Yönergesi (v-html): Veriyi düz metin değil gerçek HTML öğesi olarak işlemek için kullanılır.
- Güvenlik Uyarısı (XSS): v-html kullanıcı tarafından sağlanan verilerde kesinlikle kullanılmamalıdır; XSS açıklarına yol açabilir.

```
// Şablon Örneği
<!-- Metin İnterpolasyonu (Düz Metin) -->
<span>Mesaj: {{ rawHtml }}</span>

<!-- v-html Yönergesi (Gerçek HTML) -->
<p>
  Ham HTML çıktısı:
  <span v-html="rawHtml"></span>
</p>

<script setup>
const rawHtml = '<span style="color: red">Kırmızı</span>'
</script>
```

Nitelik Bağlantıları (v-bind) & Stenografi

HTML Niteliklerini Bağlama

- v-bind Yönergesi: HTML niteliklerini (id, class, src, disabled) reaktif bileşen durumuna bağlar.
- Kısaltılmış Kullanım (Stenografi): 'v-bind:id' yerine sadece ':id' yazılabilir.
- Aynı İsimli Stenografi (Vue 3.4+): Değişken adı nitelik adı ile aynıysa ':id' kullanımı 'id' değişkenini bağlar.
- Dinamik Güncelleme: Reaktif değişken değiştiğinde HTML niteliği anında güncellenir.

```
// v-bind Kullanım Şekilleri
<!-- Tam Sözdizimi -->
<div v-bind:id="dynamicId"></div>

<!-- Stenografi (Kısaltma) -->
<div :id="dynamicId"></div>

<!-- Aynı İsimli Stenografi (Vue 3.4+) -->
<div :id></div>

<script setup>
const dynamicId = 'container-box'
const id = 'main-wrapper'
</script>
```

Mantıksal Nitelikler & Çoklu Nitelik Bağlama

Boolean Attributes & v-bind Object

- Mantıksal (Boolean) Nitelikler: :disabled='isButtonDisabled' ifadesinde değer true ise nitelik eklenir; false/null/undefined ise kaldırılır.
- Çoklu Nitelik Bağlama: v-bind='objectOfAttrs' kullanılarak bir JavaScript nesnesindeki tüm nitelikler tek seferde öğeye aktarılır.
- Dinamik Nesne: Nesne içindeki id, class, style gibi tüm anahtarlar karşılık gelen HTML niteliklerine dönüşür.

```
// Mantıksal & Çoklu Bağlama Kodları
<!-- Mantıksal Nitelik -->
<button :disabled="isButtonDisabled">Gönder</button>

<!-- Çoklu Nitelik Bağlama -->
<div v-bind="objectOfAttrs"></div>

<script setup>
import { ref } from 'vue'

const isButtonDisabled = ref(true)

const objectOfAttrs = {
  id: 'container',
  class: 'wrapper-box',
  'data-role': 'admin'
}
</script>
```

JavaScript İfadeleri Kullanımı & Sınırlamalar

Şablon İçi JS İfadeleri

- Tek Satırlık İfadeler: {{ number + 1 }}, {{ ok ? 'YES' : 'NO' }} gibi dönüştürülebilir tam JS ifadeleri desteklenir.
- Fonksiyon Çağrıları: Şablon içerisinde bileşen metotları çağrılabilir: {{ formatDate(date) }}.
- Kısıtlamalar: İfadeler yalnızca TEK bir ifade olmalıdır. Değişken atamaları (var a = 1) veya 'if/else' blokları çalışmaz.
- Global Erişim Kısıtlaması: Şablondan yalnızca Math, Date gibi sınırlı küresel nesnelere erişilebilir.

```
// Şablon İçi İfade Örnekleri
<!-- Matematiksel İfade -->
<p>{{ number + 1 }}</p>

<!-- Üçlü (Ternary) Operatör -->
<p>{{ ok ? 'EVET' : 'HAYIR' }}</p>

<!-- Metin İşlemleri -->
<p>{{ message.split('').reverse().join('') }}</p>

<!-- Dinamik Niteliğe İfade Bağlama -->
<div :id=""></div>
```

Yönergeler, Argümanlar & Dinamik Argümanlar

Yönerge Yapısı & Dinamik Sözdizimi

- Yönergeler (Directives): 'v-' ön ekine sahip reaktif DOM davranışları tanımlayan özel niteliklerdir.
- Argümanlar: Yönerge adından sonra iki nokta ile eklenir (v-bind:href veya v-on:click).
- Dinamik Argümanlar: Köşeli parantez içinde JavaScript ifadesi kullanılarak nitelik veya olay adı dinamikleştirilir.
- Sözdizimi Kısıtlamaları: Dinamik argüman değeri 'null' olursa bağlama kaldırılır. Tırnak ve boşluk içeren karmaşık ifadeler geçersizdir.

```
// Dinamik Argüman Örnekleri
<!-- Dinamik Nitelik Argümanı -->
<a :[attributeName]="url"> Link </a>

<!-- Dinamik Olay Argümanı -->
<a @[eventName]="doSomething"> Action </a>

<script setup>
import { ref } from 'vue'

const attributeName = ref('href')
const url = ref('#')
const eventName = ref('click')
</script>
```

Değiştiriciler (Modifiers)

Yönerge Değiştiricileri Mantığı

- Değiştirici (Modifier): Yönerge adından sonra nokta '.' ile eklenen özel soneklerdir.
- .prevent Değiştiricisi: Olay tetiklendiğinde otomatik olarak event.preventDefault() çağırır.
- .stop Değiştiricisi: Olayın yukarı yayılmasını (event.stopPropagation()) engeller.
- Tuş Değiştiricileri: @keyup.enter gibi spesifik klavye tuşlarını dinlemeyi sağlar.

```
// Değiştirici Örnekleri
<!-- Formun yeniden yüklenmesini önler -->
<form @submit.prevent="onSubmit">
  <button type="submit">Gönder</button>
</form>

<!-- Tıklama olayının yayılmasını durdurur -->
<a @click.stop="doThis">Tıkla</a>

<!-- Sadece Enter tuşuna basıldığında çalışır -->
<input @keyup.enter="submitForm" />
```

Temel Reaktif Yönergeler (v-if, v-for, v-on, v-model)

Çekirdek Yönergeler

- v-if / v-else-if / v-else: Koşula göre öğeleri gerçek DOM'a ekler veya çıkarır.
- v-for: Dizileri döngüye alarak liste öğeleri oluşturur. Benzersiz ':key' kullanımı performans için zorunludur.
- v-on (@): DOM olaylarını (click, input, submit) dinler.
- v-model: Form girdisi ile reaktif değişken arasında çift yönlü veri bağlama (Two-way Data Binding) sağlar.

```
// Toplu Yönerge Örneği
<script setup>
import { ref } from 'vue'

const searchText = ref('')
const items = ref([
  { id: 1, name: 'Elma' },
  { id: 2, name: 'Armut' }
])
</script>

<template>
  <input v-model="searchText" placeholder="Ara..." />

  <p v-if="items.length === 0">Liste boş.</p>
  <ul v-else>
    <li v-for="item in items" :key="item.id">
      {{ item.name }}
    </li>
  </ul>
</template>
```

BÖLÜM SONU ÖZETİ

3. Bölüm Sonu: Template Syntax (Şablon Sözdizimi) — Özet ve Değerlendirme

Bölüm 3 Kazanımları ve Temel Saptamalar

- Vue, HTML tabanlı şablon sözdizimini son derece optimize edilmiş JavaScript render fonksiyonlarına derler.
- Metin interpolasyonu {{ }} ve nitelik bağlama v-bind (:) ile bildirimsel veri akışı sağlanır.
- Şablon içinde tek satırlık JavaScript ifadeleri ve dinamik argümanlar kullanılabilir.
- v-if, v-for, v-on (@) ve v-model yönergeleri arayüz reaktivitesini ve etkileşimini yönetir.

BÖLÜM 4

Components Basics (Bileşen Temelleri)

Bileşen tanımlama, kayıt, yeniden kullanılabilirlik, props, /emits, slotlar, dinamik bileşenler ve DOM şablon ayrıştırma uyarıları.

Bileşen Tanımlama ve Kullanımı (Lokal Kayıt)

Bileşen Mimarisi & Reusability

- Modüler UI: Arayüzü bağımsız, kapsüllenmiş ve yeniden kullanılabilir parçalara böler.
- SFC Tanımlama: .vue uzantılı dosyalarda <script setup> ile tanımlanır.
- Lokal Kayıt: 'import ButtonCounter from './ButtonCounter.vue'' şeklinde içe aktarılan bileşenler şablonda doğrudan kullanılır.
- Yeniden Kullanılabilirlik: Bir bileşen şablonda kaç kez çağrılırsa çağrılsın her kullanımda bağımsız yeni bir örnek (instance) oluşturulur.

```
// App.vue (Bileşen Kullanımı)
<script setup>
import ButtonCounter from './ButtonCounter.vue'
</script>

<template>
  <h1>Bileşen Örnekleri</h1>

  <!-- Her buton kendi bağımsız sayacını tutar -->
  <ButtonCounter />
  <ButtonCounter />
  <ButtonCounter />
</template>
```

Props ile Veri Aktarımı (Parent to Child)

Tek Yönlü Veri Akışı (Props)

- Props Nedir?: Üst (parent) bileşenden alt (child) bileşene veri aktarmak için tanımlanan özel niteliklerdir.
- definePropS() Makrosu: SFC içerisinde aktarılacak prop isimlerini tanımlamak için kullanılır.
- Tek Yönlü Veri Akışı: Props alt bileşen tarafından doğrudan değiştirilemez (read-only); veri her zaman yukarıdan aşağıya akar.
- Dinamik Prop Geçişi: v-bind (:title='post.title') ile reaktif değişkenler aktarılır.

```
// BlogPost.vue (Alt Bileşen)
<script setup>
// Props tanımlama
defineProps(['title', 'likes'])
</script>

<template>
  <div class="blog-post">
    <h4>{{ title }}</h4>
    <p>Beğeni Sayısı: {{ likes }}</p>
  </div>
</template>
```

Olay Dinleme ve Fırlatma (/ emits)

Aşağıdan Yukarıya İletişim

- Child to Parent İletişimi: Alt bileşen üst bileşendeki veriyi doğrudan değiştiremez; bunun yerine bir olay (event) fırlatır.
- () Metodu: Alt bileşen içerisinde özel olay fırlatmak için çağrılır: ('enlarge-text').
- defineEmits() Makrosu: Script içerisinde fırlatılacak olay isimlerini bildirmek için kullanılır.
- Üst Bileşende Dinleme: Üst bileşen alt bileşen üzerindeki olayı '@enlarge-text' ile dinler.

```
// BlogPost.vue ( Kullanımı)
<script setup>
defineProps(['title'])
// Fırlatılacak olayları tanımla
const emit = defineEmits(['enlarge-text'])
</script>

<template>
  <div class="blog-post">
    <h4>{{ title }}</h4>
    <!-- Butona basılınca olayı fırlat -->
    <button @click="emit('enlarge-text')">
      Metni Büyüt
    </button>
  </div>
</template>
```

Slotlar ile İçerik Dağıtımı & Dinamik Bileşenler

Slotlar & Dinamik Görünümler

- Slotlar (<slot>): Üst bileşenden alt bileşenin gövdesine esnek HTML veya şablon içeriği aktarmayı sağlar.
- Dinamik Bileşenler: <component :is='currentTabComponent'> etiketi ile sekmeli görünümlerde bileşenler arasında dinamik geçiş yapılır.
- <KeepAlive> Etiketi: Dinamik geçişlerde pasif duruma geçen bileşenin durumunu (state) hafızada korumak için sarmalama sağlar.

```
// AlertBox.vue & Dinamik Sekme
<!-- AlertBox.vue (Slot Kullanımı) -->
<template>
  <div class="alert-box">
    <strong>Hata Oluştu!</strong>
    <slot />
  </div>
</template>

<!-- Dinamik Sekme Kullanımı -->
<KeepAlive>
  <component :is="activeTabComponent" />
</KeepAlive>
```

DOM İçi Şablon Ayrıştırması Uyarıları

HTML Ayrıştırıcı Kısıtlamaları

- DOM Template Parsing: Şablon doğrudan HTML dosyası içinde yazılıyorsa tarayıcının HTML ayrıştırma kurallarına tabi olur.
- Büyük/Küçük Harf Duyarsızlığı: HTML etiketlerde büyük harfi küçük harfe dönüştürür. PascalCase bileşenler 'kebab-case' (<my-component>) yazılmalıdır.
- Kapanış Etiketleri: <my-component /> kendiliğinden kapanan etiketi DOM içinde geçersizdir; açık kapanış (<my-component></my-component>) yazılmalıdır.
- Öğe Yerleştirme Kısıtlamaları: <ul>, <table>, <select> içine sadece belirli etiketler girebilir. Çözüm: <tr is='vue:my-row'>.

```
// DOM Uyarıları Kod Örnekleri
<!-- 1. Kebab-case Kullanım Zorunluluğu -->
<!-- HTML DOM içinde: -->
<blog-post-item></blog-post-item>

<!-- 2. Kapanış Etiketi Zorunluluğu -->
<my-component></my-component>

<!-- 3. Öğe Yerleştirme (is="vue:...") -->
<table>
  <tr is="vue:my-table-row"></tr>
</table>
```

BÖLÜM SONU ÖZETİ

4. Bölüm Sonu: Components Basics (Bileşen Temelleri) — Özet ve Değerlendirme

Bölüm 4 Kazanımları ve Temel Saptamalar

- Vue bileşenleri, kullanıcı arayüzünü bağımsız ve yeniden kullanılabilir modüllere bölmeyi sağlar.
- Veri aktarımıProps ile yukarıdan aşağıya (Parent -> Child) tek yönlü olarak gerçekleştirilir.
- Alt bileşenler durum değişikliklerini ile özel olaylar fırlatarak üst bileşene bildirir.
- Slotlar esnek içerik dağıtımı sunar; DOM içi şablonlarda harf duyarsızlığı ve etiket kısıtlamalarına dikkat edilmelidir.

Genel Değerlendirme & Sonuç

Vue.js Mimarisi Saptamaları

- Esnek ve Kademeli Yapı: CDN ile basit HTML sayfalarından Vite tabanlı devasa SPA projelerine kadar sorunsuz ölçeklenir.
- Bildirimsel & Reaktif Şablonlar: Sürekli DOM manipülasyonu yazma zorunluluğunu ortadan kaldırarak geliştirici verimliliğini artırır.
- Modüler Bileşen Mimarisi: Props, Emits ve Slot yapıları sayesinde temiz, bakımı kolay ve test edilebilir kod tabanı oluşturur.
- Güçlü Ekosistem: Vite, Pinia ve Vue Router araçlarıyla modern web standartlarını eksiksiz destekler.

Web Programlama I Açısından Kazanımlar

- Frontend Mimarisi Kavrayışı: Modern JavaScript kütüphanelerinin ve framework'lerinin çalışma mantığı kavranmıştır.
- Durum Yönetimi & Reaktivite: Veri ile kullanıcı arayüzü arasındaki senkronizasyon ilkeleri öğrenilmiştir.
- Bileşen Odaklı Tasarım: Arayüzlerin küçük, kapsüllenmiş parçalara bölünerek yönetilmesi deneyimlenmiştir.
- Geleceğe Hazırlık: Kurumsal web projelerinde kullanılan endüstri standardı geliştirme araçlarına hakimiyet sağlanmıştır.

Kaynakça & Referanslar

Akademik & Teknik Referans Listesi

- 1. Sivas Cumhuriyet Üniversitesi, Mühendislik Fakültesi, Bilgisayar Mühendisliği Bölümü — Web Programlama I Ders Notları (2026).
- 2. Vue.js Resmi Dokümantasyonu — 'Hızlı Başlangıç Rehberi' (Quick Start Guide)
- 3. Vue.js Resmi Dokümantasyonu — 'Vue Uygulaması Oluşturma' (Creating a Vue Application)
- 4. Vue.js Resmi Dokümantasyonu — 'Şablon Sözdizimi' (Template Syntax)
- 5. Vue.js Resmi Dokümantasyonu — 'Bileşenlerin Temelleri' (Component Basics)
- 6. Evan You & Vue Core Team — 'Vue 3 Core Concepts & Composition API Guide', Open Source Software Documentation.